

— CURRENT STATE REVIEW · 技术与产品审计

# Pi Stuff

## 当前能力与未完成工作

一份以实际使用体验为主线、以正式代码与真实 Pi 认证为证据的现状报告。

JC Zhang

Current State Review · 2026.08.02

Locally certified prerelease

# | 目录

|    |               |    |
|----|---------------|----|
| 01 | 结论先行          | 3  |
| 02 | 现在启动 Pi, 得到什么 | 4  |
| 03 | 对话与 UI 的实际形态  | 6  |
| 04 | 四项已实现能力       | 8  |
| 05 | 工程成熟度与可信边界    | 11 |
| 06 | 还有什么没做        | 13 |
| 07 | 明确不做与暂不做      | 17 |
| 08 | 建议的后续顺序       | 19 |
| A  | 附录: 证据与术语     | 21 |

# 结论先行

Pi Stuff 已经不是概念，也不是一堆散装插件。它已经形成一条可在本机日用的第一条完整纵向切片： 同一个 Pi Package 中包含统一临时界面、破坏性操作保护、子 Agent、会话 Todo 与 BTW 旁路问答，并由当前 Pi 自动加载。但它仍是本地预发布版，离“成熟 coding agent 的完整能力层”还有明显距离。

一句话判断

基础骨架已经可靠；最核心的四个交互已能用；统一工具输出、长期记忆与恢复、工作控制、网页与外部连接仍未进入正式 Suite。

## 五个产品大块的当前成熟度

| 大块                   | 现在已经有                                                 | 下一层还缺                                           |
|----------------------|-------------------------------------------------------|-------------------------------------------------|
| ① 工作组织与 Agent 协调     | Agents、session Todo                                   | Plan/提问、后台命令、Goal、Loop/Cron                     |
| ② 监督与交互 UI           | Command Dialog、Todo 清单、Agent roster、原生 Permissions 设置 | runtime/Goal/scheduled 等临时页面、整套 keymap/settings |
| ③ 跨上下文与跨 session 连续性 | 当前 Todo/Agent run 的 session 内恢复                       | Mem0 长期记忆与明确的跨 session 召回                       |
| ④ 安全与可恢复性            | Permissions accident circuit breaker                  | rewind、冲突保护与远程副作用边界                             |
| ⑤ 可扩展能力层             | Aggregate、owned-fork 来源治理                             | 统一工具渲染、Web、MCP、默认 Skills、 /doctor               |

工程状态：278 个测试、1168 次断言、真实 Pi 0.83 Host/PTY 与离线打包认证全部通过；但六个 package 仍是 0.0.0，当前分支领先远端 11 个提交，尚未形成这一版的远端 CI 与 npm release。

后文用四个词保持边界： 可用 表示正式代码已认证， 已选 表示产品方向或 pinned fork base 已确认， 等待 表示依赖 Pi 上游接口， 不做 表示明确的产品边界。第 8 章只是建议，不冒充已确认决策。

# 现在启动 Pi，得到什么

当前机器的 Pi 设置已经指向正式仓库中的 Aggregate Package。新启动一个 Pi 进程，它会直接载入这套本地源码；不需要另开命令，也不会出现启动广告或第二套主界面。

## 启动时的实际行为

- 1. Pi 从用户 settings 读取 `../../dev/pi-stuff/packages/pi-stuff`。
- 2. Aggregate 按 UI → Permissions → Agents → Todo → BTW 的固定顺序加载。
- 3. 加载失败时 fail-fast，不会悄悄变成只有一半功能的状态。
- 4. quietStartup 已启用，因此正常启动没有 Pi Stuff banner。
- 5. 已在运行的旧 Pi 进程不会热加载；重启后才使用最新本地代码。
- 6. Extension 初始化保持纯净：不联网、不调用模型、不写文件或配置，也不启动子进程。

它是什么：由原生 Pi Host 加载的一套能力包。  
它不是什么：另一套 CLI、另一套模型循环、另一套 session 系统，或套在 Pi 外面的新 TUI。

## 六个可发布 package

| Package                         | 角色                          | 用户是否直接看到             |
|---------------------------------|-----------------------------|----------------------|
| @jczhang02/pi-stuff             | Aggregate：一个入口，按序加载全套能力     | 通常看不到                |
| @jczhang02/pi-stuff-ui          | 共享 Command Dialog、焦点与界面冲突协调 | 通过其他能力看到             |
| @jczhang02/pi-stuff-permissions | 默认无打扰的破坏性操作保护               | 危险操作或打开设置时           |
| @jczhang02/pi-stuff-agents      | 后台/前台子 Agent、roster、详情与控制   | 有子 Agent 或 /agents 时 |
| @jczhang02/pi-stuff-todo        | 当前 session 的工作清单与依赖状态       | 有当前任务时               |
| @jczhang02/pi-stuff-btw         | 不污染主对话的旁路问答                 | 输入 /btw 时            |

系统关系



图 1：蓝色突出四项用户可见 Capability；上方可见呈现与下方协调/状态层也包含 Pi Stuff 自有代码，Pi Host 仍拥有主循环与原生 TUI。

Host 与 Pi Stuff 的分工

| Pi Host 继续负责                             | Pi Stuff 负责                                |
|------------------------------------------|--------------------------------------------|
| Agent 主循环、模型与 thinking、基础工具              | 选定的工作组织、旁路问答与子 Agent 体验                    |
| session/tree/fork/resume、compaction、输入队列 | Todo session state、Agent lifecycle 与 UI 投影 |
| 原生 TUI、Settings、Package 与动态工具加载          | 共享非浮动 Dialog、危险操作保护、未来 adapters            |

# 对话与 UI 的实际形态

现在的整体 UI 理念已经落地到共同基础：**conversation-first**、**Claude Code** 风格的信息层级、**Pi 原生容器**、**无浮窗**。它不是像 GUI 那样同时铺满卡片，而是让主要对话始终占据屏幕。

## 正常屏幕

主对话 transcript

你的消息、Agent 文字、Pi 原生工具输出

✓ 已查清正式加载方式

■ 整理报告结构

□ 生成并检查 PDF

... +2 pending

输入框中的当前草稿

↓ to manage

● main

○ general-purpose    审计能力    done   · 31s

○ general-purpose    审计缺口    done   · 27s

Todo 只在有任务时出现在编辑器上方；Agent roster 只在当前 session 有直接子 Agent 时出现在编辑器下方。当前生产实现的宽屏上限是 5 个直接子 Agent，64 列及以下 4 个，另加 ↓ to manage 与 ● main；超出时显示 overflow。没有内容时，这两块完全消失。Pi Stuff 不添加常驻状态栏，也不把“健康度、memory、MCP、Goal”塞在底部。

## 最常见的临时 UI：Command Dialog

Command Dialog 是分隔线下方、占满编辑区宽度的临时交互区域。它不是 popup，也不是盖在 transcript 上的浮动卡片。/agents、/btw、Permissions 审批与原生 SettingsList 都使用这一类容器。

原来的 transcript 保持原位

Agents / BTW / Permission / Settings

当前列表、详情或回答

↑/↓ 选择 · Enter 查看 · Esc 返回

打开时会保存输入草稿，清空编辑器，隐藏 footer、working row、Todo 与 Agent roster；关闭时恢复原来的草稿和所有 chrome。权限审批是 blocking 页面：它可以暂时抢占正在看的 BTW 或 Agents 页面，审批后继续显示原页面，而不是把状态重建一遍。

对话内会出现哪些输出

| 类别           | 现在如何显示                                           | 是否进入主 transcript           |
|--------------|--------------------------------------------------|----------------------------|
| 用户消息         | Pi 原生可见分隔与正文                                     | 是                          |
| Assistant 文字 | 普通可读文本，不套大卡片                                     | 是                          |
| Pi 内建工具      | 仍由 Pi 原生输出渲染器（renderer）显示                        | 是                          |
| Agents 工具    | 紧凑回执；后台状态在 roster 与 /agents                      | 回执与直接子 Agent 摘要进入          |
| Todo 工具      | 成功调用静默；状态在上方清单                                   | 状态快照保存，但不形成噪声卡片            |
| BTW          | 只在临时 Dialog 内显示                                  | 否                          |
| 权限审批         | blocking Dialog；Allow 后只继续这次精确调用，Deny 把拒绝原因返回请求方 | Dialog 本身不进入；工具后续结果仍走原工具路径 |

已统一与尚未统一

- 已经统一：Pi Stuff 自有能力的临时页面、焦点、草稿、chrome、Todo 与 Agent roster 之间的协调。

尚未统一：所有 Read、Grep、Find、Write、Edit、Bash、测试、diff 与未来 Web/MCP 工具的普通 transcript 呈现。

已确认的未来方向是“共享输出规则 + 密度 C”：连续低影响探索形成一条语义摘要，只保留最多两个代表性子记录与 +N more；测试、写入、失败、拒绝等后果明显的操作保持独立。完整输出进入非浮动 Tool Details Dialog，避免 Pi 的全局 expand-all 在长会话中一次性展开所有历史内容。

截图说明

本报告没有把仓库中的 PNG 当作当前产品截图。现有 82 张图全部是 Claude 黑盒参考或真实 Pi 0.83 上的可丢弃 UI 原型；它们证明方向可行，但不是正式 Aggregate 的生产画面。这里使用结构示意与真实测试结论，避免制造“已经实现”的错觉。

## 04 · IMPLEMENTED CAPABILITIES

# 四项已实现能力

以下不是路线图，而是当前正式 package 已经提供的行为。每一项都从“用户怎么用、屏幕上看到什么、状态存在哪里、边界是什么”解释。

## Agents：默认后台的子 Agent

当你说“让多个 Agent 并行研究”时，主 Agent 可以调用一个 `subagent` 工具，启动单个或多个子 Agent。默认在后台运行，主 Agent 继续当前工作；确实需要立即等待结果时才用 `foreground`。

### 当前能做

- `single / parallel`；`fresh context / fork parent context`；共享 `cwd` / 独立 `Git worktree`。
- 为每个 Agent 指定 `cwd`、`model`、`thinking`、`skills`、`turn budget` 与 `tool budget`。
- 查询 `status`、发送 `steer`、`stop`、`resume`；详情页查看 `transcript` 与 `partial result`。
- 内置 `general-purpose` Agent 开箱可用，并继承项目 `context` 与 `Skills`。
- 发现 `package`、用户目录和项目目录中的 Agent 定义；项目定义优先级最高。

### 屏幕上是什么样

平时只在编辑器下方显示直接子 Agent 的一行 `roster`。输入框为空时按 `Down` 进入，`Up/Down` 选择，`Enter` 打开详情，`x` 停止或移除，`Esc` 回到编辑器。`/agents` 打开当前 `session` 的完整列表与详情；这不是 Agent Library，也不是跨 `session` Fleet。

### 固定保护

- 最大嵌套深度 3；同时运行上限 20；单 `session` 总启动数上限 200。
- 后台完成后，只有直接子 Agent 的紧凑摘要回到主对话；更深层 `transcript` 留在详情页。
- `worktree` 只有在“干净、无新 `commit`、状态可确认”时自动删除；存在任何改动或不确定性就保留。
- 所有深度的危险操作审批都转发到根 `session`；根 `session` 不可用则快速拒绝。

当前小边界：`foreground` Agent 可从 UI 停止，但不能在这个页面内 `steer`；界面会明确告诉用户未执行，不会伪装成功。

## Todo：当前 session 的 Agent 工作清单

Todo 主要给 Agent 自动使用，而不是让用户维护第二个项目管理系统。Agent 获得

`TaskCreate`、`TaskGet`、`TaskList`、`TaskUpdate` 四个工具；成功调用不在 `transcript` 生成大量卡片。



## 当前能做

- 稳定字符串 ID，永不复用；状态为 pending、in\_progress、completed、deleted。
- 依赖、owner、metadata；拒绝自依赖、缺失依赖和循环依赖。
- 最多显示 5 行，再加一行 overflow；最近完成、进行中、可开始、被阻塞按可读顺序排列。
- Ctrl+Shift+T 在完整列表与一行 Next: ... 之间切换；Agent 更新任务后自动展开。
- 全部完成后保留 5 秒再隐藏；任务数据本身不删除。

## 状态如何恢复

Todo 没有另建数据库。每次工具结果带版本化快照，Pi 将它保存在原生 session transcript 中；reload、compaction 或切换 session tree 分支时，从当前分支重放。因此它是当前会话的工作状态，不是跨 session 项目 backlog，也不替代 Beads。

当前缺口：没有 /todo 或完整的用户操作页面；用户不能在 UI 中直接新增、拖动、重排或删除任务。当前是 Agent-facing 工具加 bounded checklist。

## BTW：主任务不停的旁路问答

输入 /btw 为什么这里要用 worktree? ，Pi 会在主 Agent 继续工作时打开一个全宽旁路问答。它使用当前模型与 provider 认证，但明确设置 tools=[], 所以不会读文件或运行命令。

## 隔离保证

- 读取当前主会话有效上下文，包括 compaction summary；去掉尚未完成的 assistant 消息。
- 历史图片替换为“图片已省略”，避免旁路请求携带旧图。
- 问题与答案都不进入主 transcript，也不会影响主 Agent 后续上下文。
- 关闭 BTW 只取消旁路调用，不取消主 Agent。
- 上下文超长时自动收缩，provider 仍 overflow 时再缩小一次重试，并显示明确提示。

## 历史与快捷键

直接输入 /btw 可重新打开本 session 的成功历史，最多 20 条。Left/Right 切历史，Up/Down 或 Ctrl+P/Ctrl+N 滚动，c 复制答案，x 清除较早历史，Esc 关闭。历史只存在当前 Pi 进程；退出即消失。失败只在当前 Dialog 给出错误，不写入成功历史。BTW 内没有连续追问输入框，每次命令是一问一答。

当前技术边界：Pi 0.83 没有“完全复用 Host 模型调用、同时不写 transcript”的公开接口。BTW 复用了当前模型、provider、认证、base URL、headers 与 thinking，但不继承完整生命周期 hooks 和全部重试/传输设置。Beads ps-5cb.3 专门等待这个上游接口；不会为此 fork Pi 或破坏隔离。

## Permissions: 默认不打扰, 只拦明显事故

默认模式是 `unrestricted`。读取、编辑、测试、普通 shell, 以及当前 `cwd` 内目标明确的普通删除都不询问。它不是“每一步先问你”, 而是在高置信度破坏性操作出现时拒绝或询问一次。

### 永远拒绝

- 删除 `/`、`$HOME`、当前 `cwd`、Git worktree root、`.git` 或包含这些目录的祖先。
- 用变量、命令替换、brace、glob 等方式让删除目标无法静态确认。
- 用 `sudo`、`busybox`、`eval`、嵌套 shell 等包装已识别的破坏动作。
- 破坏性形态无法安全解析时, 拒绝并要求改写为目标明确的命令。

### 只审批这一次精确调用

- 删除 `cwd` 外、但目标明确的文件或目录。
- `git reset --hard`、非 `dry-run` `git clean`、`git restore`。
- 覆盖路径内容的 `git checkout`、`force/discard` `git switch`、`git stash drop/clear`。

审批页只有 `Allow this exact call once` 与 `Deny`。它显示请求者、完整命令、`cwd`、操作类型、目标与原因; 不会把一次允许变成永久宽权限。终端太小时暂停审批并提示调大, `Esc` 始终可拒绝。

### 设置与边界

`/permissions` 在共享 Command Dialog 内打开 Pi 原生 `SettingsList`, 可调整 `permission mode`、`review log`、`debug logging`、`double-press`。高级规则可以收紧工具、路径、MCP、Skill 与外部目录权限, 但项目内容和子 Agent 不能放松全局 `deny` 或 `circuit breaker`。默认启动不会创建 `config` 或 `log`; 只有用户实际修改设置或开启记录后才需要持久化。

必须准确理解: Permissions 是 coding agent 的事故断路器, 不是 OS sandbox。它能覆盖经过认证的直接 shell/Git 破坏形态, 但无法证明任意脚本、MCP server 或第三方案程序内部没有等价副作用。

当前已认证的破坏命令族: 注册的 Suite shell tool 中的 `rm`、`rmdir`、`unlink`、`find -delete`, 以及前述 Git discard 形态。当前不承诺覆盖 Python/任意脚本内部删除、shell function/alias、独立安装的 MCP/Extension 副作用、symlink race, 或 `move`、`overwrite`、`chmod/chown`、`mount`、`block-device` 等其他系统操作。

# 工程成熟度与可信边界

当前实现的工程证据很强，但证据边界也必须说清：它证明“本地源码在指定环境中可用且可打包”，不证明“已经发布”或“所有平台兼容”。

## 2026-08-02 本地完整验证

| 检查                                     | 结果                              |
|----------------------------------------|---------------------------------|
| Bun tests                              | 278 pass / 0 fail               |
| Test files                             | 56                              |
| Assertions                             | 1168                            |
| TypeScript projects                    | 5 / 5 通过                        |
| Biome                                  | 0 error; 24 warnings; 102 infos |
| Knip / generated drift / public safety | 全部通过                            |
| Package certification                  | Certified with Pi 0.83.0        |

## 认证不是简单 typecheck

package verifier 会离线生成 artifact、校验 hash 与 allowlist、阻止开发文件混入、执行 publish dry-run，分别安装 Agents、BTW、Permissions、Todo 及精确 runtime dependencies，再用 Pi RPC 检查命令与 Task 工具。最后从 Aggregate tarball 运行真实 Host/PTY 路径，并检查许可证、UPSTREAM provenance 与共享 UI coordinator 的唯一身份。

## 真实 Pi / PTY 覆盖

| 能力          | 真实验证                                                                                                          | 准确边界                              |
|-------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------|
| Agents      | 源码 Aggregate PTY: 100×32 与 64×28; 提取后 tarball gate 再跑 64×28; 覆盖并发、roster、详情、草稿恢复、completion、nested permission | 覆盖最完整                             |
| BTW         | 100×32、64×28; 主任务并发、tools=[]、transcript 隔离、滚动与恢复                                                              | 完整 Host 旁路调用接口仍缺                  |
| Permissions | blocking Dialog、长证据滚动、exact-call allow/deny、grandchild→root                                                   | 不是 OS sandbox                     |
| Todo        | 真实 Host/Aggregate load、四工具、replay、依赖、renderer/integration tests                                               | 没有独立 automated real-PTY test file |

## 兼容性范围

- 已认证: Pi standalone host 0.83.0、Linux x64、Bun 1.3.14、TypeScript 5.9.3。
- 尚未宣称: 其他 Pi 版本、macOS、Windows。
- 真实 narrow acceptance: 64×28; 普通 acceptance: 100×32。

## fork 与来源治理

| 能力             | 采用的 owned fork base                               | 来源规则                       |
|----------------|---------------------------------------------------|----------------------------|
| Permissions    | @gotgenes/pi-permission-system@24.0.0 · 776ebc... | MIT; 保留 license / UPSTREAM |
| Agents         | pi-subagents@0.38.0 · 89de10...                   | MIT; 去除上游浮窗 UI             |
| Todo           | @juicesharp/rpiv-todo@2.3.1 · 75823a...           | MIT; 保留 state engine, 重做呈现 |
| BTW            | @juicesharp/rpiv-btw@2.3.1 · 75823a...            | MIT; 保留隔离机制, 适配当前 Host     |
| UI / Aggregate | Pi Stuff 原生实现                                     | 公开 Pi API; 无第二 Host        |

产品行为可以参考 Claude Code、tintinweb 与用户旧配置，但代码不能来自未获授权的 Claude source，也不能从 jczhang02/pi-agent 复制。未来采用成熟 package 时同样必须先固定公开版本、许可证与 commit，再 fork 到 Pi Stuff；不是运行时依赖上游作者的 UI 与决策。

# | 还有什么没做

“没做”不等于“没有想法”。大部分方向已有明确的产品边界和 pinned fork 选择，但尚未进入 suite、尚未通过 Pi 0.83 真实原型门槛，也没有生产 UI。以下按五个产品大块分组。

## A. 工作组织与 Agent 协调

Agents 与 session Todo 已经完成；下一层是让 Agent 在执行前、执行中、后台运行与定时唤醒时都有清楚且彼此不混淆的控制方式。

| 做完后你会怎么用                                                                | 现在到哪                                                                                                                       | 还缺什么                                                                                       |
|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Plan + 结构化提问<br>Agent 先只读研究并展示计划；确实需要选择时，用清楚的选项页询问，再交回执行                | 已选 pinned owned forks:<br>@narumitw/pi-plan-mode ·<br>aceaf779... 与 @juicesharp/<br>rpiv-ask-user-question ·<br>f28d733... | 还没有只读阶段、计划批准/交接、选择题页面、命令或真实 Pi 验收；question package 的 catalog/version 来源对应仍需在 vendoring 时归一 |
| 后台命令 / Monitor<br>长测试转入后台后主对话继续；在 /tasks 查看输出、停止任务或观察 command/WebSocket | 选定要 fork 的基础 pi-background-tasks@0.7.6 ·<br>4a359cb...，并决定与 subagent 共用运行视图                                                | 还没有后台 Bash、输出文件、停止/重新查看、Monitor、合并通知与统一运行服务                                                |
| Goal<br>给出完成条件，Agent 每轮判断是否达成；未达成时在不可关闭的极端安全上限内继续                       | 选定 @narumitw/pi-goal@0.31.0 · 3ad2...，但仍须通过安全 gates                                                                        | 还没有完成判断、自动继续、恢复、时间/轮次/token 上限或 /goal                                                      |
| Loop / Cron<br>在当前 session 中“十分钟后再检查”或按 cron 定期唤醒；忙时只合并一次提醒             | 选定 @davidroman/pi-loop@0.3.5 · 98cdf...，并确认它不与 Todo 或运行任务混为一体                                                              | 还没有调度、到期、忙时合并、一次性/cron 命令或管理页面                                                             |

B. 监督与交互 UI

| 做完后你会怎么用                                                                       | 现在到哪                                                                                                                                                | 还缺什么                                                                |
|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| 统一工具输出<br>Agent 连续读文件时只<br>显示一条摘要与最多两<br>条代表记录；进入 Tool<br>Details 后只展开所选<br>输出 | 已选 <code>owned fork @mobrienv/pi-tidy-<br/>tools@0.4.1</code> , tag <code>pi-tidy-tools-<br/>v0.4.1 / commit 4b251377...</code> ；密度 C<br>与非浮动详情页已确认 | 还没有 package；普通 Pi<br>工具仍用原生样式；自动分<br>组、长输出上限、性能缓存<br>与快捷键未实现        |
| 整套设置与键位<br>在 Pi 原生设置页开关能<br>力、调整输出密度与快<br>捷键                                   | 容器方向已确认；当前 Todo 使用<br><code>Ctrl+Shift+T</code> , 最终 UI target 是<br><code>Ctrl+T</code>                                                             | 目前只有 Permissions 设<br>置；还需解决 Pi 原有键位冲<br>突，并提供功能开关、密度<br>与共享 keymap |

还有一项窄屏差异需要归一：已接受的 64×28 统一 UI 原型只保留 `main + 2` 个子 Agent；当前生产 roster 是 `main + 4` 。本报告前文写的是当前真实实现，未来落统一 work surface 时要明确迁移到哪一个值。

C. 跨上下文与跨 session 连续性

| 做完后你会怎么用                                                                                | 现在到哪                                                                                                                                                                                                                   | 还缺什么                                                                |
|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Mem0 长期记忆<br>开启新 session 时仍<br>能召回本项目的约<br>定； <code>/memory</code> 可查<br>看、修改、删除与关<br>闭 | 已选 <code>embedded owned-fork</code> 方向：以官方<br><code>@mem0/pi-agent-plugin@0.1.4</code><br><code>integration slice</code> 与 <code>mem0ai/mem0</code><br><code>TypeScript core ( 50bdaae...</code> ) 为基础；数据<br>留本机、按项目隔离 | 还没有 package、存储或<br>页面；20 条记忆与<br>100/1k/10k 规模的可丢弃<br>Pi/Bun 原型尚未完成 |

现有 Todo replay 和 Agent transcript 只保证当前 session / run 的状态可恢复，不能在新 session 中自动召回项目经验。

D. 安全与可恢复性

| 做完后你会怎么用                                                                            | 现在到哪                                                                                                                      | 还缺什么                                                                                         |
|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>rewind / recovery</code><br>每次用户请求到来时自动<br>留恢复点；出错后可恢复<br>文件、对话或两者，并能<br>撤销恢复 | Permissions 已提供当前事故断路器；<br><code>rewind</code> 选定 <code>arpagon/pi-<br/>rewind@0.5.0 · 91611ad...</code> 作为<br>要 fork 的基础 | 还没有冲突保护、undo、非 Git<br>路径、文件/对话恢复或子<br>Agent 恢复；外部 API、部署等<br>远程副作用不能由 <code>rewind</code> 撤销 |

E. 可扩展能力层

| 做完后你会怎么用                                                           | 现在到哪                                                   | 还缺什么                                                                  |
|--------------------------------------------------------------------|--------------------------------------------------------|-----------------------------------------------------------------------|
| 网页读取<br>直接让 Agent 搜索并读取网页、PDF 或 GitHub，同时显示来源和明确失败原因               | 产品层已选 pi-web-access@0.17.1；只保留搜索、抓取与文档读取核心             | 还没有 package；来源对应、真实 Pi/Bun 原型，以及防恶意 URL 访问内网、无限重定向、压缩炸弹和巨型 PDF 的限制未完成 |
| MCP / 外部工具<br>按需连接外部 server；第一次使用时看得见它要访问什么，并只加载需要的工具              | 选定 repo nicobailon/pi-mcp-adapter v2.17.0 · 82c631d... | 还没有 package；兼容性、信任、登录授权、权限范围、按需工具目录与设置未实现                             |
| 能力目录<br>通过 /doctor 看哪些能力已连接、哪些未配置、为什么暂时降级                          | 可见性方向已确认，并与按需连接配合                                      | 还没有统一目录、降级说明或连接状态页面                                                   |
| 默认 Skills<br>用 /batch、/simplify、/debug、/code-review、/verify 启动常见工作 | 已确定为后续默认 Skills set，不另造执行引擎                            | 还没有正式资源、package 或验收                                                   |

F. 唯一明确的上游等待项

ps-5cb.3 只追踪 BTW 的一个公开接口缺口。现在的 BTW 已可用，但 Pi 0.83 还不能让这次旁路请求完整经过 Host 的模型调用生命周期、上下文 hooks、重试与传输设置，同时又保证不写入主对话。正确处理是等待 Pi 提供公开接口；不是 fork Pi，也不是让 BTW 污染主 transcript。

G. 发布与项目管理仍缺的工作

| 缺口            | 为什么重要                                                                                                 | 完成标志                                                 |
|---------------|-------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| 远端证明          | 11 个本地提交没有对应的 GitHub Actions 运行                                                                       | push 后当前 commit 的完整 CI 通过                            |
| 正式版本          | 六个 package 仍为 0.0.0, 4 个 changesets 未消费                                                               | 范围冻结、版本提升、npm release 可安装                            |
| 实施账本          | 当前 15 个 Beads: 13 closed、ps-5cb in progress、ps-5cb.3 open; 未来能力主要埋在 epic 长评论                          | 每项能力有独立 Beads、依赖、owner 与 acceptance                  |
| 决策归一          | BTW 的未来 f promotion 与“本 tranche 不做 fork”、更大的 Auto/SRT (自动判断与持续进程隔离) 目标与当前 accident breaker 之间仍有历史记录冲突 | 用新的 canonical Beads 明确 supersede 关系, 再建立实现 issue     |
| GitHub mirror | 当前产品 Beads 没有 external_ref, 尚未形成对应 GitHub issue mirror                                                | 按 push-only policy 发布选定 issue, 并保留 Beads 为 canonical |
| 文档一致性         | 部分历史文档仍写 Hermes 或旧四块模型                                                                                | 标为 superseded, 并指向 Mem0 与当前五块模型                      |
| 证据保全          | 若干临时 evidence 路径已消失                                                                                   | 正式 repo 留存可复查的审计、原型与结论                               |
| 生产画面          | 82 张现有 PNG 均是参考或原型                                                                                    | 从正式 Aggregate 生成 production PTY capture              |



# 明确不做与暂不做

成熟不是把所有相邻功能都塞进来。以下项目被明确排除，或被放到独立 package / 外部能力层；不能在“未完成”列表里把它们误判为欠债。

## 冻结的长期边界

| 项目                           | 决定   | 原因                                               |
|------------------------------|------|--------------------------------------------------|
| LSP                          | 完全移除 | 不是默认、optional 或 gate；未来需要时作为独立 Pi Package 重新讨论   |
| 第二套 Host / CLI / TUI         | 不做   | Pi 已拥有主循环、session、模型、工具与 TUI                     |
| 浮窗 / overlay / 常驻 statusline | 拒绝   | 破坏 conversation-first；所有复杂交互进入非浮动 Command Dialog |

## 当前默认 scope 外

| 项目                                    | 当前决定  | 未来条件                               |
|---------------------------------------|-------|------------------------------------|
| Agent View / Fleet                    | 当前不内建 | 跨 session supervisor 需要单独产品决策      |
| Agent Teams / saved chains / role zoo | 当前不内建 | 不得静默扩张成 orchestration 平台           |
| Dynamic Workflows / Ultracode         | 当前不内建 | 若未来讨论，仍受“无第二 workflow runtime”边界约束 |
| 共享/发布/托管 artifact                     | 当前不内建 | 涉及 cloud、数据与 sharing，需另立产品范围       |

## 暂不内建，但可由外部能力达到

- 可交互浏览器 / computer use：暂不作为默认 Suite 能力；可由 MCP、外部 tool 或独立 package 接入。
- Advisor：涉及 provider、数据去向与成本边界，留待未来明确决策。
- 任意第三方工具的统一显示：Pi 0.83 没有“只替换显示、不接管执行”的通用接口。Pi Stuff 只承诺内建工具与 owned forks；独立安装的第三方 extension 只有在可包装执行或主动采用共同显示规则时才能统一。

## 几个容易误解的当前边界

- BTW 当前是 process-local history、一次一问、没有 promotion。epic 曾把 f 显式提升到新 session 写入未来 target，但 ps-5cb.2 又明确本 tranche 不做 fork；实现前需要一条新的 canonical 决策归一，而不能把它写成当前能力。
- Todo 与 Beads 是两层：Todo 是当前 session 的执行清单；Beads 是项目决策、长期事项与 acceptance。
- Agents 的 /agents 是当前 session 运行视图，不是 Claude 旧版 Agent Library Hub。
- Permissions 不是 sandbox；外部工具接入后仍要通过独立 trust、scope 与 process adapter 设计。
- Beads hook 未安装是仓库 policy 下的预期状态，不是当前实现故障。

## 新能力进入 Pi Stuff 的判断顺序

1. Pi Host 是否已经拥有这项能力？如果已经拥有，优先复用，不再造一套。
2. 它是否是多数 coding session 都会使用的能力？如果只服务特殊工作流，优先做独立 package。
3. 是否存在许可证与来源清楚、下载和维护证据足够的成熟 package？存在时先审计，再 owned fork。
4. 它能否遵守共享输出规则、非浮动 Dialog、原生 Settings 与统一安全边界？不能时先修共同机制，不绕开体系。

### 产品边界

Pi Stuff 要成为“成熟 Pi 的能力层”，而不是“又一个 coding-agent 平台”。它应补足 Pi 缺少的产品能力，同时让 Pi 继续拥有 Host。

## 08 · RECOMMENDED SEQUENCE

# | 建议的后续顺序

下面是基于已确认产品模型的展开顺序：先补每天都能看到的核心 UI，再补长期连续性，最后连接外部世界。它是本报告的建议，不自动成为新决策；确认后应拆成 Beads。

## 0. 先让决策系统变得可执行

把 ps-5cb 中每个已选能力拆成独立 Beads，写清作为 fork 基础的上游版本、必须保留、必须删除、Pi 0.83 prototype gate、UI acceptance、依赖与明确 out-of-scope。同步标记过时研究文档，消除 Mem0/Hermes 与旧产品分块冲突。

## 1. 完成统一工具输出

这是当前最显眼的体验断层，也是后续 Web/MCP/Plan/Background tools 的共同基础。以已选定的 @mobrienv/pi-tidy-tools@0.4.1 owned fork 为内核，先建立 Suite 输出 contract，再覆盖 Pi Stuff 包装的内建工具；实现密度 C、长输出 hard cap、按宽度缓存与非浮动 Tool Details。先不承诺透明接管任意第三方工具。

## 2. 并行做 Mem0 可丢弃原型

Memory 的风险主要不在 UI，而在本地存储、项目隔离、检索质量、规模与故障退化。在正式 pi-stuff repo 的隔离 branch/worktree 中建立可丢弃 prototype，验证 20 条真实测试记忆、同名 repo 隔离、100/1k/10k 条延迟、同义召回与文字检索回退、重启、更新/删除范围、embedding 失败与零外传。通过后再转为正式 Capability Package；失败则回退到 Hermes 方向。

## 3. 补齐工作与对话控制

按共享 UI 原语实现 Plan、结构化提问与后台 shell/Monitor。Todo、runtime /tasks、Goal、Cron/Loop、Beads 必须各自拥有独立的数据、命令与生命周期，只共享 ID、事件、错误、安全和 UI 基础，不能合并成一个含义模糊的“任务系统”。

## 4. 加入 recovery，再决定 Goal / Loop 的生产顺序

rewind 直接保护真实开发文件，优先于复杂自动 continuation。Goal 与 Loop 的 fork base 已选，但都是 GO-WITH-GATES；必须先完成超时、token/turn ceiling、busy coalescing、session scope、恢复与安全矩阵，避免形成无人看管的无限运行。

## 5. 接入 Web 与 MCP

先统一安全与呈现，再接 capability：URL/redirect/DNS/解压/PDF 限额、untrusted-content boundary、OAuth/scope、lazy catalog、明确 degraded errors。网页读取已经在产品层选定，仍需通过来源对应与真实 Pi/Bun gates；交互浏览器继续由外部工具提供。

## 6. 形成第一个公开版本

在一组连贯能力通过本地与远端 CI 后，清理 lint debt、补 production capture、确认支持矩阵、消费 changesets、发布 npm。不要为了“有版本号”提前宣称未来能力已完成。

| 顺序  | 目标             | 完成标志                                        |
|-----|----------------|---------------------------------------------|
| 0   | Beads 与文档归一    | 每项能力都有 owner、gate、acceptance、依赖             |
| 1   | 统一工具 UI        | 真实 Pi 100×32 / 64×28；密度 C；Tool Details；性能通过 |
| 2   | Mem0 prototype | 隔离、质量、规模、重启、降级、零外传均通过                       |
| 3-4 | 工作控制与 recovery | 共享 Dialog、权限链、数据与生命周期分离、恢复矩阵                |
| 5   | Web / MCP      | 安全边界、连接可见性、统一输出规则                           |
| 6   | 首个公开 release   | 远端 CI、版本、npm、支持矩阵、production capture        |

### 下一件最值得做的事

把“统一工具输出”拆成正式 Beads 并实现第一条生产级输出渲染；同时让 Mem0 prototype 在隔离分支后台验证。前者改善每天都能看到的体验，后者尽早消除长期架构风险。

## A · APPENDIX

## | 附录：证据与术语

## A.1 主要证据入口

- [Repository README](#)：项目状态、包入口与开发命令。
- [Compatibility](#)：Pi 0.83.0、Linux x64、Bun 与 TypeScript 边界。
- [suite.json](#)：Aggregate 的五个 capability package 与加载顺序。
- [Shared UI README](#)：Command Dialog coordinator 的公开行为。
- [Agents README](#)、[Todo README](#)、[BTW README](#)、[Permissions README](#)。
- [Mem0 fit research](#)：当前 memory 方向与 prototype gates。
- [Tool UI early research](#)：记录早期价值与限制；其中“暂定候选”状态已过时，最终 fork 选择见 Beads ps-5cb。
- [Package verifier](#)：离线 tarball 与真实 Host/PTY 认证路径。
- Beads ps-5cb：产品能力集与决策日志；ps-5cb.3：BTW Host 旁路调用接口。

## A.2 术语表

Host: Pi 本身。它拥有 Agent 主循环、模型、session、基础工具、TUI、Settings 与 Package loader。

Capability Package: 只增加一项明确能力的 Pi Package，例如 Agents 或 BTW。

Aggregate Package: 用户只配置一次，它按固定顺序加载整套 capability packages。

Command Dialog: 分隔线下方、非浮动、临时替换编辑区的交互页面；关闭后恢复草稿与原界面。

owned fork: 从固定公开版本、commit 与许可证出发，把代码纳入 Pi Stuff，由本项目拥有后续 UI、修复、测试和发布，而不是原样依赖上游产品决策。

session Todo: 本轮对话中的执行清单；不等于项目 issue tracker。

Beads: Pi Stuff 的长期决策、issue、依赖与 acceptance 记录，是正式项目账本。

locally certified prerelease: 本地源码已通过完整认证并被当前 Pi 使用，但尚未形成 npm 公开 release，也没有当前提交的远端 CI 证明。

## A.3 审计口径

- “已实现”要求正式 packages/ 中存在、Aggregate 能加载、测试与认证通过。
- “已选”表示 Beads 已确认产品方向或 pinned fork base，但正式 Capability Package 尚未实现，不能描述为现在可用。
- “原型”只证明设计或机制可行，不等于生产代码。
- “明确不做”是产品边界，不计入完成度欠账。